



# Sajilo Damak

*Ramro Sewa, Sajilo Jeevan.*

**Complete Feature Manual** — every feature, exactly how it works, and what it uses.

Version 1.0 · October 2026 · Damak Municipality, Jhapa, Nepal

Covers the web application (customer, professional and administration portals) and its API.

## Contents

1. Product at a glance — portals, roles, rules
2. Customer portal features
3. Professional portal features
4. Administration portal features
5. Cross-cutting systems (coverage, money, OTP, rewards, notifications, consent, security)
6. Maps and location subsystem
7. Technology inventory — what everything uses
8. Data model summary
9. API endpoint catalog
10. Running it — environments, seed data, demo logins

## 1. Product at a glance

---

Sajilo Damak is a **Damak-only** on-demand service marketplace. Customers book verified local professionals; professionals work assigned jobs; administrators run verification, dispatch, money and content. It is a single build serving four experiences routed by hostname (`www / clients / pros / admin` subdomains in production, path routing on localhost).

| Portal              | Audience | Home route | Sees                                              |
|---------------------|----------|------------|---------------------------------------------------|
| Public site         | Visitors | /          | Discovery, marketing, policies, sign-in           |
| Customer portal     | CUSTOMER | /dashboard | Bookings, quotes, rewards, addresses, support     |
| Professional portal | WORKER   | /worker    | Jobs, requests, earnings, availability, documents |
| Administration      | ADMIN    | /admin     | 14-section control centre (see §4)                |

**Non-negotiable rules:** (1) every booking is server-validated against the Damak boundary; (2) professionals enter by admin invitation only and must be verified + activated before any assignment; (3) money is integer paisa with snapshotted commissions and append-only ledgers; (4) jobs close only with the customer's one-time code; (5) role enforcement happens on the backend — the portals are navigation, not security.

## 2. Customer portal features

### 2.1 Registration, sign-in and onboarding tutorial

**How it works.** Public registration creates **CUSTOMER** accounts only — roles are never client-assignable (the register handler hard-codes the CUSTOMER role). Sessions are 7-day HS256 JWTs in an `HttpOnly, SameSite=Lax, Secure-in-production`, host-only cookie, re-validated with fresh role lookup on every request. New accounts land on a 5-step tutorial ( `/welcome` ): how-it-works → profile (name/phone) → first Damak address with map pin → push opt-in → rewards/referral intro; completion persists in `users.onboarded`, and logins funnel un-onboarded users there once. Signup requires an explicit Terms + Privacy checkbox.

**Uses:** `POST /api/auth/register|login|logout`, `GET|PATCH /api/auth/me`, `PATCH /api/auth/me/onboarding`; `bcryptjs` (cost 12), `jose` JWT, `Zod` validation, React Hook Form; pages `Signin.tsx` (sliding auth card), `Onboarding.tsx`.

### 2.2 Service discovery

**How it works.** Searchable directory over admin-configured services only. Filters for text, category, max price and minimum rating; sorts for popular, price and rating. Ratings and job counts are computed live from completed bookings and genuine reviews — nothing is fabricated. Quote-type services show “Custom quote” instead of a price.

**Uses:** `GET /api/categories`, `GET /api/services`, `GET /api/services/:id` (service + eligible verified pros + reviews); pages `Home.tsx` (hero, ticker, tiles, stats from `GET /api/stats`), `Services.tsx`, `ServiceDetail.tsx`.

## 2.3 Direct booking (Mode A)

**How it works.** Four steps — address → slot (next 7 days, 4 daily windows) → details + payment + rewards → review. On submit the server: rejects past slots; snapshots the category commission rate onto the booking; re-validates the Damak address and open ward; applies loyalty redemption (balance-checked inside the same transaction); creates the booking ( `pending` ), its first timeline event and a pending payment row — all atomically with a unique human `BK-XXXX` number.

**Uses:** `POST /api/bookings` (transactional, idempotent booking numbers); page `Book.tsx` ; money math in `utils/money.ts` .

## 2.4 Quote requests (Mode B)

**How it works.** For complex jobs: title, category, detailed description, photos, preferred window, ward/landmark (+ optional pin). The request is routed to eligible verified pros with the customer's identity and exact address masked (ward + description only). Pros propose fixed prices; high-value proposals wait for admin approval per platform policy; the customer compares and accepts one — acceptance mints a `confirmed` booking with the pro assigned and an assignment record.

**Uses:** `POST /api/quotes/requests` , `GET /api/quotes/requests/mine|open` , `POST .../proposals` , `POST /api/quotes/proposals/:id/approve|accept` ; tables `quote_requests` , `quote_proposals` ; pages `QuoteNew.tsx` , Dashboard proposals.

## 2.5 Tracking, reschedule, rebook, disputes

**How it works.** Every booking carries a full status timeline ( `pending` → `awaiting-worker` → `confirmed` → `en-route` → `in-progress` → `awaiting-confirmation` → `completed` , plus `cancelled/disputed` ) with actor + timestamp per step — history is never rewritten. Customers can reschedule (future slots, blocked mid-job except by admins), cancel free before confirmation, raise disputes, review completed jobs (one per booking), and rebook any past service in one tap.

**Uses:** `GET /api/bookings/mine|:id` (history + assignments), `POST .../transition` (role-aware state machine in `utils/transitions.ts` ), `PUT .../slot` , `POST /api/reviews` ; page `Track.tsx` .

## 2.6 Loyalty, referrals and rewards

**How it works.** Points per Rs 100 of completed spending, milestone bonuses, 100-pts-to-Rs-50 redemption at checkout, and a referral program (code at signup → both sides paid on the referee's first completed job; self-referral rejected). All balances derive from an append-only ledger with a one-earn-per-booking database constraint; refunds auto-reverse awards.

**Uses:** `reward_ledger`, `referral_codes`, `referral_uses` ; `GET /api/rewards/mine` , `GET /api/referrals/mine` ; commission/reward posting inside the completion transactions; pages `Rewards.tsx` , Dashboard loyalty card.

## 2.7 Addresses, support, notifications

**How it works.** Up to 5 Damak-only addresses (city normalized, ward parsed or explicit, closed wards rejected, optional GPS pin inside the municipality box). Support tickets with threaded replies and reopen-on-follow-up. In-app notification inbox with unread tracking plus opt-in web push (see §5).

**Uses:** `/api/addresses` , `/api/tickets` , `/api/notifications(+read-all)` ; pages `Profile.tsx` , `Support.tsx` , `Notifications.tsx` ; header `NoticeBell` .

## 3. Professional portal features

### 3.1 Invitation and verification

**How it works.** Two doors, one gate. (a) Email invite: admin creates an invite (only a hash is stored), the worker claims it via a token link, sets phone + password, and lands `awaiting-documents`. (b) Office flow: the customer signs up normally, submits certificates physically, and the admin sends the invite *to their profile* — they accept in-app from their dashboard and land in the pro portal. Verification states are `draft` → `awaiting-documents` → `under-review` → `verified` → `rejected|suspended` with an immutable per-change record. Activation additionally requires skill selection and is blocked unless verified.

**Uses:** `worker_invites`, `worker_profiles`, `verification_records`, `verification_documents`, `worker_services`, `worker_availability`; `/api/admin/workers/invite|invite-user|verify|activate`, `/api/auth/worker/invite/check|accept`, `/api/worker/invites/mine|accept`; pages `Invite.tsx`, `Worker.tsx` documents tab.

### 3.2 Jobs, requests and completion codes

**How it works.** The portal answers four questions: today's schedule, open requests matching the pro's skills (self-accept enforces the same eligibility as admin assignment), upcoming jobs, and next actions. Expandable job cards show instructions, customer contact (tap-to-call), payment, estimate, map and route. When work ends the pro issues the completion code (job moves to awaiting-confirmation, code delivered to the customer only); the customer reads it out, the pro submits it — correct code completes the job, posts commission + rewards + referral bonuses. Wrong codes count attempts (5 max), codes expire in 10 minutes and are hashed at rest.

**Uses:** `GET /api/worker/jobs|me|reviews`, `POST /api/bookings/:id/otp/issue|verify`, `booking_otps` table, `utils/otp.ts` (crypto RNG, sha256); page `Worker.tsx`.

### 3.3 Availability, earnings, quotes, routing

**How it works.** Weekly open/closed days control slot offers. Earnings show a per-booking ledger (total, commission, keep, settled state) plus settlement history — finalized figures are never editable by pros. Quote requests appear masked; proposals state approval needs. Each job offers in-app OSRM routing (distance/ride-time chip) from the pro's GPS fix plus an OSM directions link.

**Uses:** `GET|PUT /api/worker/availability`, `GET /api/worker/earnings`, `commission_ledger`, `settlements`, `cash_collections`, `lib/geo.ts` `fetchRoute` (OSRM), `LiveMap` component.

## 4. Administration portal features

---

The control centre has 14 sections: **Overview** (linked stat cards, unassigned alerts, verification queue, disputes, latest bookings) · **Dispatch** (status filters, per-booking drawer with timeline, assign/reassign with eligibility + recorded reason, status correction, live multi-stop job map, aging badges) · **Workers** (state filters, email + in-profile invites, document review, verify/reject/suspend, skill selection + activation gate) · **Customers** (search, full file, restrict/lift) · **Quotes** (requests with proposals, approvals) · **Catalog** (inline price/duration/active editing with historical snapshots preserved, new services + categories) · **Finance** (commission ledger with settled filter, cash collections, settlement recorder, payment lookup + refunds with reward reversal) · **Rewards** (live rules + settings editor) · **Reviews** (moderation with delete) · **Support** (reply, resolve/reopen) · **Broadcast** (role-audience in-app + push) · **Reports** (date range, totals, per-day bars, top services, payments split, pro leaderboard) · **Audit** (every sensitive action with actor/action/detail) · **Settings** (coverage wards editor, platform rules).

**Uses:** GET

/api/admin/overview|bookings|workers|customers|customers/:id|quotes|services-all|ledger|reviews|tickets|audit|settings|wards|reports/summary|payments and the matching write routes (see §9); capability-checked by `role_permissions` ( `worker.verify`, `finance.settle...` ); page `Admin.tsx` with sidebar + slide-in drawers.

## 5. Cross-cutting systems

### 5.1 Damak-only coverage

**How it works.** A single pure module ( `utils/coverage.ts` , unit-tested) normalizes city names, detects Damak mentions, and parses ward numbers from free text. Addresses, bookings and quote requests are all gated server-side: non-Damak cities rejected with an explanation, closed wards rejected, services must list Damak, and pins must fall inside the municipality box. Coverage lives in data ( `coverage_wards` , `settings.platform` ) — new towns are rows + an admin toggle, never a code change.

### 5.2 Money: commissions, cash, online payments, refunds

**How it works.** All money is integer paisa. Category commission rates (basis points) snapshot onto each booking; worker/category overrides live in `commission_rules` . Completion posts one immutable ledger row (total/commission/keep/rate). Cash collected on-site is recorded per booking and reconciled via payout/collection settlements — never silently deducted. eSewa/Khalti run a sandbox initiate → idempotent callback → server-side verify flow keyed on unique provider refs; frontend success is never trusted. Refunds support partial amounts and auto-reverse reward points. Reports always separate gross, collected, commission and refunds.

### 5.3 Notifications: in-app guaranteed, push as the fast lane

**How it works.** Every event writes an in-app row (the guaranteed channel). Web push (VAPID) fans out best-effort per device subscription and audience, with 404/410 pruning — it can never break business logic. The completion code itself stays in-app (lock-screen safe); pushes only nudge. Clients get a header bell with unread badge, a full notification centre, and an opt-in toggle with self-test buzz.

**Uses:** `notifications` , `push_subscriptions` ; `services/push.ts` ( `web-push` ), `/api/push/*` , custom `public/sw.js` , `lib/push.ts` , `NoticeBell` , `PushToggle` , `Notifications.tsx` .

## 5.4 Consent, privacy, legal

**How it works.** A real consent manager (banner with accept-all / reject-optional / customize, per-category preference centre, versioned re-prompts) gates optional tracking — the inventory lists only technologies actually in use. Records persist server-side. Separate Terms, Privacy, Cookie and Cancellation documents with contents sidebar, versions and contact paths.

**Uses:** `consent_records`, `POST|GET /api/consent*`, `lib/consent.ts`, `CookieConsent.tsx`, `Legal.tsx`.

## 5.5 Security and audit

**How it works.** Helmet headers, CORS locked to the app origin, tiered rate limits, Zod validation on every write, per-request role re-fetch, capability permissions, audit log on all sensitive actions, bcrypt-hashed passwords, private verification documents, `X-Robots-Tag: noindex` on admin APIs, open-redirect-safe navigation, and idempotent, retry-safe writes (unique constraints + transactions).

# 6. Maps and location subsystem

| Piece        | What it is                                                                                                         | What it uses                                                               |
|--------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| MapPicker    | Pin dialog: Damak-biased search, tap/drag pin, streets + satellite, locate-me, live reverse label, Damak bbox lock | Leaflet + react-leaflet, Nominatim + Photon geocoding, browser geolocation |
| MiniMap      | Non-interactive preview with OSM attribution; marker only for real pins                                            | Leaflet, OSM tiles                                                         |
| LiveMap      | Multi-stop dispatch/pro map with popups, pro position, OSRM route polyline, smart fit-bounds                       | Leaflet, OSRM demo server                                                  |
| Checkout map | Always-visible job-location panel; saved pin, Photon area geocoding, or Damak overview with honest states          | MiniMap + MapPicker + booking pin override                                 |
| Autofill     | Pin pick fills address text + ward (editable after); GPS fix fills gaps                                            | reverseLabel, parseWardFromText                                            |
| Storage      | Optional lat/lng on addresses, bookings, quote requests; Nepal CHECK constraints + Damak enforcement               | PostgreSQL, migration 003                                                  |

Tiles: OpenStreetMap (free, keyless, attributed). Satellite: Esri World Imagery (free, keyless, attributed). Routing: OSRM demo server (free, keyless). Geocoding: Nominatim + Photon (both free/keyless, debounced, Damak-filtered).

# 7. Technology inventory

## Frontend (single build, four portals)

| Technology                       | Used for                                                                                               |
|----------------------------------|--------------------------------------------------------------------------------------------------------|
| React 19 + TypeScript + Vite     | UI, type-safe development, dev server + production build                                               |
| Tailwind CSS v4 + design tokens  | Styling; pine/marigold M3-informed theme in <code>index.css</code>                                     |
| React Router                     | Portal routes + role guards ( <code>Protected.tsx</code> ) + host routing ( <code>lib/host.ts</code> ) |
| React Hook Form + Zod            | All forms with schema validation                                                                       |
| Lucide icons                     | Iconography incl. per-category tiles                                                                   |
| Leaflet + react-leaflet          | All maps (picker, previews, live)                                                                      |
| Push API + custom service worker | Opt-in web push display                                                                                |

## Backend (API-first for a future Android app)

| Technology                                         | Used for                                                                 |
|----------------------------------------------------|--------------------------------------------------------------------------|
| Node.js + Express + TypeScript                     | REST API ( <code>/api/*</code> ), stateless for VPS or container scaling |
| PostgreSQL 16 ( <code>pg</code> pool)              | All data; schema + numbered idempotent migrations; integer-paisa money   |
| jose (HS256 JWT)                                   | Host-only session cookies                                                |
| bcryptjs                                           | Password hashing (cost 12)                                               |
| Zod                                                | Request validation with field errors                                     |
| helmet / cors / cookie-parser / express-rate-limit | Security headers, origin lock, cookies, tiered throttling                |
| web-push (VAPID)                                   | Push fan-out with endpoint pruning                                       |
| tsx (dev) / tsc (build) + node:test                | Dev runner, typechecks, 21 unit tests                                    |

## External services

| Service                         | Used for                               | Status model                   |
|---------------------------------|----------------------------------------|--------------------------------|
| OpenStreetMap tiles + Nominatim | Map tiles, search bias, reverse labels | Live, free/keyless, attributed |

|                           |                                         |                                                           |
|---------------------------|-----------------------------------------|-----------------------------------------------------------|
| Photon (Komoot)           | Street/locality search Nominatim misses | Live, free/keyless                                        |
| Esri World Imagery        | Satellite layer                         | Live, free/keyless, attributed                            |
| OSRM demo server          | Pro driving routes + ride estimates     | Live, free/keyless, best-effort                           |
| eSewa / Khalti            | Online payments                         | Sandbox flow built; live keys pending merchant onboarding |
| Netlify + VPS (Nginx/PM2) | Frontend hosting + API/database hosting | Documented in <code>docs/DEPLOY.md</code>                 |

*Nothing above is claimed operational beyond what is implemented: payments stay sandbox until merchant keys exist; SMS/push-provider channels attach later at the notify service.*

## 8. Data model summary

| Group              | Tables                                                                                                                                                                                                                     |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Identity & access  | <code>users</code> , <code>roles</code> , <code>permissions</code> , <code>user_roles</code> , <code>role_permissions</code> (+ <code>users.onboarded</code> )                                                             |
| Professionals      | <code>worker_invites</code> (email + in-profile), <code>worker_profiles</code> , <code>verification_records</code> , <code>verification_documents</code> , <code>worker_services</code> , <code>worker_availability</code> |
| Catalog & coverage | <code>categories</code> , <code>services</code> , <code>coverage_wards</code> , <code>settings(platform)</code>                                                                                                            |
| Customers & jobs   | <code>addresses</code> , <code>bookings</code> (+ward/lat/lng), <code>booking_events</code> , <code>assignments</code> , <code>booking_otps</code> , <code>quote_requests</code> , <code>quote_proposals</code>            |
| Money              | <code>payments</code> , <code>cash_collections</code> , <code>refunds</code> , <code>commission_rules</code> , <code>commission_ledger</code> , <code>settlements</code>                                                   |
| Loyalty            | <code>reward_ledger</code> , <code>referral_codes</code> , <code>referral_uses</code>                                                                                                                                      |
| Trust & comms      | <code>reviews</code> , <code>notifications</code> , <code>push_subscriptions</code> , <code>support_tickets</code> , <code>ticket_messages</code>                                                                          |
| Governance         | <code>consent_records</code> , <code>audit_log</code>                                                                                                                                                                      |

## 9. API endpoint catalog

---

### Auth & session

POST /api/auth/register|login|logout · GET /api/auth/me · PATCH /api/auth/me|me/onboarding · POST /api/auth/worker/invite/check|accept

### Public

GET /api/categories|services|services/:id|reviews|wards|settings|stats

### Account (signed in)

/api/addresses (GET|POST|DELETE) · /api/reviews · /api/tickets(+/:id/messages) · /api/rewards/mine · /api/referrals/mine · /api/notifications(+/:id/read|read-all) · /api/consent(+/latest) · /api/worker/invites/mine|accept

### Bookings

POST /api/bookings · GET /api/bookings/mine|:id · POST /api/bookings/:id/transition|assign · PUT /api/bookings/:id/slot · POST /api/bookings/:id/otp/issue|verify

### Quotes

POST /api/quotes/requests · GET /api/quotes/requests/mine|open · POST .../:id/proposals · POST /api/quotes/proposals/:id/approve|accept

### Professional

GET /api/worker/me|jobs|earnings|availability|reviews|documents/mine · PUT /api/worker/availability · POST /api/worker/documents

### Administration (role + capability checked, noindex headers)

GET /api/admin/overview|bookings|workers|customers|customers/:id|quotes|services-all|ledger|reviews|tickets|audit|settings|wards|reports/summary|payments · POST .../workers/invite|invite-user|verify|activate · POST .../customers/:id/restrict · POST .../categories|services · PUT .../services/:id|wards|settings · POST .../commission-rules|settlements|refunds · POST .../tickets/:id/reply|status · DELETE .../reviews/:id · POST .../notifications/broadcast

### Payments & push

GET /api/payments/providers · POST /api/bookings/:id/cash-collect · POST /api/payments/:id/esewa/initiate|esewa/callback|khalti/initiate|verify · GET /api/push/vapid-key · POST /api/push/subscribe|unsubscribe|test · GET /api/push/devices

## 10. Running it

---

```
createdb sajilo
npm --workspace backend run db:migrate # schema + migrations 001-006
npm --workspace backend run db:seed   # roles, settings, catalog, admin
npm --workspace backend run db:seed-demo # demo users, jobs, quotes
npm --workspace backend run dev       # API on :4001
npm --workspace frontend run dev      # app on :5173
```

**Demo logins** (local seed only): customer `gita@demo.local / Demo1234!` · professional `bijay@demo.local / Demo1234!` · admin `admin@sajilo.local / ChangeMe123!` (rotate before any public launch).

*Repository: the project source of truth. Online payments stay in sandbox until merchant keys are configured; external notification channels beyond web push attach later without changing business logic — the API is versionable for the future Android app.*